

14

...

...

Since decimal numbering system is a base 10 numbering system any number in it is constructed using some combination of digits 0 to 9. This seems perfectly natural. However, the choice of 10 as a base is quite arbitrary, having its origin possibly in the fact that man has 10 fingers. It is very easy to use other bases as well. For example, if we wanted to use base 8 or octal numbering system, which uses only eight digits (0 to 7), here's how the counting would look like:

0

1

2

3

4

5

6

7 last available digit

10 start using a new column

11

12

...

...

Similarly, a hexadecimal numbering system has a base 16. In hex notation, the ten digits 0 through 9 are used to represent the values zero through nine, and the remaining six values, ten through fifteen, are represented by symbols A to F. The hex digits A to F are usually written in capitals, but lowercase letters are also perfectly acceptable. Here is how the counting in hex would look like:

0

1

2
3
4
5
6
7
8
9
A
B
C
D
E
F last available digit
10 start using a new column
11
...
...

Many other numbering systems can also be imagined. For example, we use a base 60 numbering system, for measuring minutes and seconds. From the base 12 system we retain our 12 hour system for time, the number of inches in a foot and so on. The moral is that any base can be used in a numbering system, although some bases are convenient than others.

The hex numbers are built out of hex digits in much the same way the decimal numbers are built out of decimal digits. For example, when we write the decimal number 342, we mean,

3 times 100 (square of 10)
+ 4 times 10
+ 2 times 1

Similarly, if we use number 342 as a hex number, we mean,

3 times 256 (square of 16)

- + 4 times 16
- + 2 times 1

Relation Between Binary and Hex

As it turns out, computers are more comfortable with binary numbering system. In a binary system, there are only two digits 0 and 1. This means you can't count very far before you need to start using the next column:

0
1 last available digit
10 start using a new column
11
...
...

Binary numbering system is a natural system for computers because each of the thousands of electronic circuits in the computer can be in one of the two states—on or off. Thus, binary numbering system corresponds nicely with the circuits in the computer—0 means off, and 1 means on. 0 and 1 are called bits, a short-form of binary digits.

Hex numbers are used primarily as shorthand for binary numbers that the computers work with. Every hex digit represents four bits of binary information (Refer Figure D.1). In binary numbering system 4 bits taken at a time can give rise to sixteen different numbers, so the only way to represent each of these sixteen 4-bit binary numbers in a simple and short way is to use a base sixteen numbering system.

Suppose we want to represent a binary number 11000101 in a short way. One way is to find its decimal equivalent by multiplying each binary digit with an appropriate power of 2 as shown below:

$1 * 2^7 + 1 * 2^6 + 0 * 2^5 + 0 * 2^4 + 0 * 2^3 + 1 * 2^2 + 0 * 2^1 + 1 * 2^0$

which is equal to 197.

Hex	Binary	Hex	Binary
0	0000	8	1000
1	0001	9	1001
2	0010	A	1010
3	0011	B	1011
4	0100	C	1100
5	0101	D	1101
6	0110	E	1110
7	0111	F	1111

Figure D.1

Another method is much simpler. Just look at Figure D.1. From it find out the hex digits for the two four-bit sets (1100 and 0101). These happen to be C and 5. Therefore, the binary number’s hex equivalent is C5. You would agree this is a easier way to represent the binary number than to find its decimal equivalent. In this method neither multiplication nor addition is needed. In fact, since there are only 16 hex digits, it’s fairly easy to memorize the binary equivalent of each one. Quick now, what’s binary 1100 in hex? That’s right C. You are already getting the feel of it. With a little practice it is easy to translate even long numbers into hex. Thus, 1100 0101 0011 1010 binary is C53A hex.

As it happens with many unfamiliar subjects, learning hexadecimal requires a little practice. Try your hand at converting some binary numbers and vice versa. Soon you will be talking hexadecimal as if you had known it all your life.

E ***ASCII Chart***

There are 256 distinct characters used by IBM compatible family of microcomputers. Their values range from 0 to 255. These can be grouped as under:

Character Type	No. of Characters
Capital letters	26
Small-case Letters	26
Digits	10
Special Symbols	32
Control Character	34
Graphics Character	128
Total	256

Figure E.1

Out of the 256 character set, the first 128 are often called ASCII characters and the next 128 as Extended ASCII characters. Each ASCII character has a unique appearance. The following simple program can generate the ASCII chart:

```
main()  
{  
    int ch ;  
  
    for ( ch = 0 ; ch <= 255 ; ch++ )  
        printf ( "%d %c\n", ch, ch ) ;  
}
```

This chart is shown on the following page. Out of the 128 graphic characters (Extended ASCII characters), there are characters that are used for drawing single line and double line boxes in text mode. For convenience these characters are shown in Figure E.2.

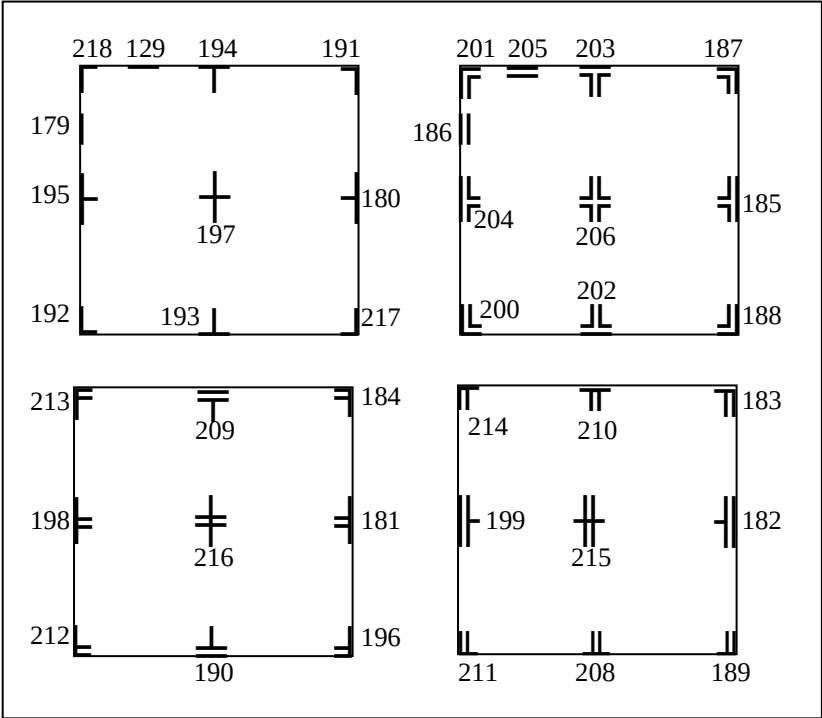


Figure E.2

Value	Char	Value	Char	Value	Char	Value	Char	Value	Char	Value	Char
0		22	▬	44	,	66	B	88	X	110	n
1	☺	23	↕	45	-	67	C	89	Y	111	o
2	☹	24	↑	46	.	68	D	90	Z	112	p
3	♥	25	↓	47	/	69	E	91	[	113	q
4	♦	26	→	48	0	70	F	92	\	114	r
5	♣	27	←	49	1	71	G	93	]	115	s
6	♠	28	┐	50	2	72	H	94	^	116	t
7	●	29	↔	51	3	73	I	95	`	117	u
8	■	30	▲	52	4	74	J	96		118	v
9	○	31	▼	53	5	75	K	97	a	119	w
10	◼	32		54	6	76	L	98	b	120	x
11	♂	33	!	55	7	77	M	99	c	121	y
12	♀	34	"	56	8	78	N	100	d	122	z
13	♫	35	#	57	9	79	O	101	e	123	{
14	🎵	36	\$	58	:	80	P	102	f	124	
15	☀	37	%	59	;	81	Q	103	g	125	}
16	▶	38	&	60	<	82	R	104	h	126	~
17	◀	39	'	61	=	83	S	105	i	127	^m H
18	↕	40	(	62	>	84	T	106	j	128	Ç
19	!!	41	)	63	?	85	U	107	k	129	ü
20	¶	42	*	64	@	86	V	108	l	130	é
21	§	43	+	65	A	87	W	109	m	131	â

Value	Char	Value	Char	Value	Char	Value	Char	Value	Char	Value	Char
132	ä	154	Û	176	⌠	198	⌦	220	■	242	≥
133	à	155	¢	177	⌡	199	⌧	221	▀	243	<
134	å	156	£	178	⌢	200	⌨	222	▁	244	┌
135	c	157	¥	179	⌣	201	〈	223	▂	245	┐
136	ê	158	₣	180	⌤	202	〉	224	α	246	÷
137	ë	159	₤	181	⌥	203	⌫	225	β	247	≈
138	è	160	₥	182	⌦	204	⌬	226	Γ	248	°
139	ï	161	í	183	⌧	205	⌭	227	π	249	•
140	î	162	ó	184	⌨	206	⌮	228	Σ	250	·
141	ì	163	ú	185	〈	207	⌯	229	σ	251	√
142	Ä	164	ñ	186	〉	208	⌰	230	u	252	η
143	Å	165	Ñ	187	⌫	209	⌱	231	τ	253	²
144	É	166	ª	188	⌬	210	⌲	232	Φ	254	■
145	æ	167	º	189	⌭	211	⌳	233	θ	255	
146	Æ	168	¿	190	⌮	212	⌴	234	Ω		
147	ô	169	¬	191	⌯	213	⌵	235	δ		
148	ö	170	¬	192	⌰	214	⌶	236	∞		
149	ò	171	½	193	⌱	215	⌷	237	ø		
150	û	172	¼	194	⌲	216	⌸	238	€		
151	ù	173	ì	195	⌳	217	⌹	239	∩		
152	ÿ	174	«	196	⌴	218	⌺	240	≡		
153	Ö	175	»	197	⌵	219	⌻	241	±		

F Helper.h File

```

LRESULT CALLBACK WndProc ( HWND, UINT, WPARAM, LPARAM ) ;

HINSTANCE hInst ; // current instance

/*  FUNCTION: InitInstance ( HANDLE, int
    PURPOSE: Saves instance handle and creates main window
    COMMENTS: In this function, we save the instance handle in a global
               variable and create and display the main program window.
*/
BOOL InitInstance ( HINSTANCE hInstance, int nCmdShow, char* pTitle )
{
    char classname[ ] = "MyWindowClass" ;
    HWND hWnd ;

    WNDCLASSEX wcex ;
    wcex.cbSize      = sizeof ( WNDCLASSEX ) ;
    wcex.style       = CS_HREDRAW | CS_VREDRAW ;
    wcex.lpfnWndProc  = ( WNDPROC ) WndProc ;
    wcex.cbClsExtra   = 0 ;
    wcex.cbWndExtra   = 0 ;
    wcex.hInstance    = hInstance ;
    wcex.hIcon        = NULL ;
    wcex.hCursor      = LoadCursor ( NULL, IDC_ARROW ) ;
    wcex.hbrBackground = ( HBRUSH )( COLOR_WINDOW + 1 ) ;
    wcex.lpszMenuName = NULL ;
    wcex.lpszClassName = classname ;
    wcex.hIconSm      = NULL ;

    if ( !RegisterClassEx ( &wcex ) )
        return FALSE ;

    hInst = hInstance ; // Store instance handle in our global variable

    hWnd = CreateWindow ( classname, pTitle,
                        WS_OVERLAPPEDWINDOW,
                        CW_USEDEFAULT, 0, CW_USEDEFAULT, 0, NULL,
                        NULL, hInstance, NULL ) ;

    if ( !hWnd )

```

```
        return FALSE ;

    ShowWindow ( hWnd, nCmdShow ) ;
    UpdateWindow ( hWnd ) ;

    return TRUE ;
}
```

G *Boot Parameters*

The disk drives in DOS and Windows are organized as zero-based drives. That is, drive A is drive number 0, drive B is drive number 1, drive C is drive number 2, etc. The hard disk drive can be further partitioned into logical partitions. Each drive consists of four logical parts—Boot Sector, File Allocation Table (FAT), Directory and Data space. When a file/directory is created on the disk, instead of allocating a sector for it, a group of sectors is allocated. This group of sectors is often known as a **cluster**. How many sectors together form one cluster depends upon the capacity of the disk. As the capacity goes on increasing, so also does the maximum cluster number. Accordingly, we have 12-bit, 16-bit or 32-bit FAT. In a 12-bit FAT each entry is of 12 bits. Since each entry in FAT represents a cluster number, the maximum cluster number possible in a 12-bit FAT is 2^{12} (4096). Similarly, in case of a 16-bit FAT the maximum cluster number is 2^{16} (65536). Also, for a 32-bit FAT the maximum cluster number is 2^{28} (268435456. Only 28 of the 32 bits are used in this FAT). All FAT systems are not supported by all versions of DOS and Windows. For example, the 32-bit FAT system is supported only in Win 95 OSR2 version or later. There are differences in the organization of contents of Boot Sector, FAT and Directory in FAT12/FAT16 system on one hand and FAT32 on the other.

In Chapter 19 Figure 19.6 we saw the breakup of the contents of the boot sector of a 12-bit FAT. Given below are the contents of a boot sector of 16-bit FAT and a 32-bit FAT.

Description	Length	Typical Values
Jump instruction	3	EB3C90
OEM name	8	MSWIN4.1
Bytes per sector	2	512
Sectors per cluster	1	64
Reserved sectors	2	1
Number of FAT copies	1	2
Max. Root directory entries	2	512
Total sectors	2	0
Media descriptor	1	F8
Sectors per FAT	2	256
Sectors per track	2	63
No. of sides	2	255
Hidden sectors	4	63
Huge sectors	4	4192902
BIOS drive number	1	128
Reserved sectors	1	1
Boot signature	1	41
Volume ID	4	4084677574
Volume label	11	ICIT
File system type	8	FAT16

Figure G.1

Let us now take a look at the 32-bit FAT system’s boot sector contents. These are shown in Figure G.2.

Description	Length	Typical Values
Jump instruction	3	EB5890
OEM name	8	MSWIN4.1
Bytes per sector	2	512
Sectors per cluster	1	8
Reserved sectors	2	51
Number of FAT copies	1	2
Root directory entries	2	0
Total sectors	2	0
Media descriptor	1	F8
Sectors per FAT	2	0
Sectors per track	2	63
No. of sides	2	255
Hidden sectors	2	63
High word of hidden sectors	4	63
Huge sectors	4	4192902
High word of huge sectors	2	4192902
Sectors per FAT	2	4095
High word of sectors per FAT	2	4095
Drive description flag	2	0
File system version	2	0
Root directory starting cluster	2	2
High word of root directory starting cluster	2	2
File system information sector	2	1
Back up boot sector	2	6
Reserved	6	0

continued...

...continued

BIOS drive number	1	128
Reserved	1	0
Boot signature	1	41
Volume ID	4	649825316
Volume label	11	ICIT
File system type	8	FAT32

Figure G.2

There are significant changes in the contents of the boot sector of a 32-bit FAT system. The entries ‘Number of hidden sectors’ and ‘Huge sectors’ have now been made 4-byte entries. The first two bytes contain the low word of the value, whereas, the next two bytes contain the high word value.

The number of sectors per FAT in a 32-bit file system is likely to exceed what can be accommodated in two bytes. Hence the entry ‘Sectors per FAT’ for a disk with a 32-bit file system would typically have a value 0. The value of ‘Sectors per FAT’ is now stored as a 4-byte entity, with the similar arrangement of low word and high word as discussed earlier.

The boot sector of a 32-bit FAT system also has new entries like ‘Drive description flag’, ‘File system version’ ‘Starting cluster number of the root directory’, ‘Sector number of the file system information sector’, and the sector number of the ‘Backup boot sector’.

The ‘Drive description flag’ is a two-byte entity. Bit 8 of this flag indicates whether or not the information written to the active FAT will be written to all copies of the FAT. The low four bits of this entry contains the 0-based FAT number of the active FAT. These bits are meaningful only if bit 8 is set.

In the entry ‘File system version number’ the high byte contains the major version number, whereas, the low byte contains the minor version number.

The entry ‘File system information sector’ contains a value indicating the sector number where the file system information is present. This file system information consists of the fields shown in Figure G.3.

Description	Length
File system signature	4
Total number of free clusters	4
Sector number of the next free cluster	4
Reserved	6

Figure G.3

The entry ‘File information sector’ contains a value 0FFFFh if there is no such sector. The entry ‘Backup boot sector’ contains a value 0FFFFh if there is no backup boot sector. Otherwise this value is any non-zero value less than the reserved sector count.

H Linux Installation

This appendix gives the steps that are to be carried out for installing Red Hat Linux 9.0. In addition I have also indicated a few commands that are necessary to compile and execute the programs given in Chapters 20 and 21. Follow the steps mentioned below to carry out the installation.

- (a) Configure the system to boot from CDROM drive.
- (b) Insert the first CD in the drive and boot the system from it.
- (c) Hit 'Enter' key when the 'boot' prompt appears.
- (d) Select the 'Skip' option in the "CD Found" dialog box.
- (e) Click on the 'Next' button in the 'Welcome' screen.
- (f) Click on the 'Next' button in the 'Language selection' screen.
- (g) Click on the 'Next' button in the 'Keyboard' screen.
- (h) Click on the 'Next' button in the 'Mouse Configuration' screen.
- (i) Select the 'Custom' option in the 'Installation Type' screen and then click on the 'Next' button.
- (j) Click on the 'Next' button in the 'Disk Partitioning Setup' screen.
- (k) Select the 'Keep all partitions and use existing free space' option in the 'Automatic Partitioning' screen and then click on the 'Next' button. Ignore any warnings generated by clicking on the 'OK' button.
- (l) Click on the 'Next' button in the 'Boot loader configuration' screen.
- (m) Click on the 'Next' button in the 'Network configuration' screen.
- (n) Click on the 'Next' button in the 'Firewall configuration' screen.
- (o) Click on the 'Next' button in the 'additional language support' screen.
- (p) Select a suitable option in the 'Time zone offset' screen and click on the 'Next' button.
- (q) Type a password for the root account in the 'Set root password' screen and then click on the 'Next' button.
- (r) Click on the 'Next' button in the 'Authentication configuration' screen.

- (s) In the 'Package group selection' screen make sure that you select the following options—X window system, K desktop environment, Development tools, GNOME software development and then click on the 'Next' button.
- (t) Select 'No' option in the 'Boot diskette creation' screen
- (u) Click on the 'Next' button in the 'Graphical Interface (x) configuration' screen.
- (v) Click on the 'Next' button in the 'Monitor configuration' screen.
- (w) In the 'Customize graphical configuration' screen select the 'Graphical' option and then click on 'Next' button.
- (x) Once the system restarts configure the system to boot from Hard Disk.

Using Red Hat Linux

For logging into the system enter the username and password and select the session as KDE (K Desktop Environment). Once you have logged in, to start typing the program use the following menu options:

KMenu | Run Command

A dialog would now pop up. In this dialog in the command edit box type **KWrite** and then click on the Ok button. Now you can type the program and save it.

To compile the program you need to go the command prompt. This can be done using the following menu option.

KMenu | System Tools | Terminal

Once at the command prompt you can use the **gcc** compiler to compile and execute your programs. You can launch another instance of the command prompt by repeating the step mentioned above.